

MEDIATEK

MT8668 Yocto Secure Boot User Manual

版本: 1.0
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	高峰	正式版

目录

版本记录 2

目录 3

图片目录 3

表格目录 3

1 Secure Boot 4

1.1 概述 4

1.1.1 名词解释 4

1.1.2 BootRom 4

1.2 安全启动 5

1.2.1 安全启动检查流程 5

1.3 下载代理认证 (DAA) 6

1.4 安全特性配置 7

1.4.1 生成密钥对 7

1.4.2 开启安全启动 8

1.4.3 编译软件 9

1.4.4 独立远程签名环境 9

1.5 签名 DA 10

1.5.1 生成 dakey.h 10

1.5.2 签名 DA 11

1.5.3 生成 Authfile 11

附件一 附加条款 12

图片目录

图 1-1.安全启动检查流程 6

表格目录

表 1-1. 名词解释 4

1 Secure Boot

1.1 概述

本文档旨在提供 MT8668 SoC 及其配套 SDK 中安全启动功能的概览。在该产品的设计和部署周期中，功能和特性可能会发生变化，这些变化将在本文档的后续版本中记录。此外，本文档还介绍了如何启用安全特性。

1.1.1 名词解释

表 1-1. 名词解释

缩略词	全称及释义
DA	Download Agent 下载代理 (包括 DA_BR 和 DA_PL)
DAA	Download Agent Authentication 下载代理认证
DRAM	Data Random Access Memory 动态随机存取内存
eFuse	Electronic Fuse 电子保险丝
eMMC	Embedded MultiMedia Card 嵌入式多媒体卡
EVB	Evaluation Board 开发板
LK	Little Kernel
NVM	Non-Volatile Memory 非易失性内存
ROM	Read-only Memory 只读存储器
SBC	Secure Boot Check 安全启动检查
SDK	Software Development Kit 软件开发包
SRAM	Static Random Access Memory 静态随机存取存储器
TEE	Trusted Execution Environment 可信执行环境
UART	Universal Asynchronous Receiver/Transmitter 通用异步收发器
USB	Universal Serial Bus 通用串行总线
USBDL	USB Download USB 下载

1.1.2 BootRom

BROM (BootROM) 是 SoC ROM 内的软件，无法修改。它是应用处理器执行的第一个软件。主要工作总结如下：

- 对 SoC 进行基础的硬件配置（例如设置 PLL，时钟等），以便启动系统。
- 从启动设备 UFS/eMMC/NAND/...加载第一阶段引导程序以启动系统。
- 与主机 PC 端的工具通信，以加载下载代理（DA）软件，用于为没有软件或软件损坏的外部 Flash 进行镜像下载。
- 使用多个安全启动检查密钥，对第一阶段引导程序或下载代理进行安全启动检查。

1.2 安全启动

1.2.1 安全启动检查流程

安全启动检查流程用于从硬件信任根创建可信执行链。可以通过烧录 eFuse 中的 SBC_EN 来启用它。每次系统从上电复位中恢复时都会执行此流程。图 1-1 展示了基本的安全启动检查流程。详细描述如下。

1. 上电复位后，BROM 使用以下流程来验证非易失性存储器（NVM，例如 eMMC 或 NAND）中的安全启动检查（SBC）公钥（SBC_PUBK）。BROM 依次使用来自 eFuse 的公钥哈希（SBC_PUBK_HASH）。
 - (1) 从 eFuse 读取安全启动检查公钥（SBC_PUBK）的哈希值（SBC_PUBK_HASH）。
 - (2) 从 NVM 读取 SBC 公钥（SBC_PUBK）。
 - (3) 计算来自（2）的数据的哈希值。
 - (4) 检查来自（1）和（3）的数据是否相同。
2. BROM 加载并验证 First-loader，这是第一阶段的引导加载程序。首先，BROM 将 First-loader 从 NVM 读取到 SoC 的 SRAM 中，并使用安全启动检查公钥（SBC_PUBK）来认证 First-loader。如果 First-loader 验证成功，将执行 First-loader。验证方法使用 SHA256 计算哈希值，RSA（2048 位）验证签名（后续会支持 RSA3072）。First-loader 包含 key cert, content cert 和 image content，具体过程如下：
 - (1) 使用 SBC_PUBK 来校验 First-loader key cert。
 - (2) 如果 First-loader key cert 校验成功，用 key cert 中的 image pub key 和 content cert 中的 image pub key 进行比较。
 - (3) 如果(2)一致，则使用 image pub key 去校验 First-loader content cert。
 - (4) 校验成功，最后计算出 First-loader image 的 hash 值，与 content cert 里存的 hash 值进行比较。

注意：

- 如果外部存储器是 NAND 闪存，BROM 支持第二份（副本）First-loader。当 BROM 无法加载第一份（副本）First-loader 时，BROM 会尝试加载/认证第二份（副本）First-loader。

3. 在第一阶段引导加载程序之后，引入了 fitimage 来进行后续的校验。包括 Kernel 和 TEE 等镜像。
4. First-loader 从 NVM 把 Linux Kernel 加载程序加载到非安全的 DRAM 区域并验证。它使用嵌入在 First-loader 中的 VERIFIED 公钥来认证校验 kernel FIT 签名，并检测 Fit 镜像中各个子镜像的 HASH 值。验证方法使用 SHA256 计算哈希值，RSA（2048 位 or 3072 位）和 MTK 或 PSS 填充进行验证。
5. First-loader 从 NVM 把 TEE 加载程序加载到非安全的 DRAM 区域并验证。它使用嵌入在 First-loader 中的 VERIFIED 公钥来认证校验 TEE FIT 签名，并检测 Fit 镜像中各个子镜像的 HASH 值。验证方法使用 SHA256 计算哈希值，RSA（2048 位 or 3072 位）和 MTK 或 PSS 填充进行验证。
6. 在 Linux 系统执行后，可以通过 Linux 内核中，MediaTek 提供的 rootfs_check 功能来实现 Kernel 对只读系统的抽样校验。

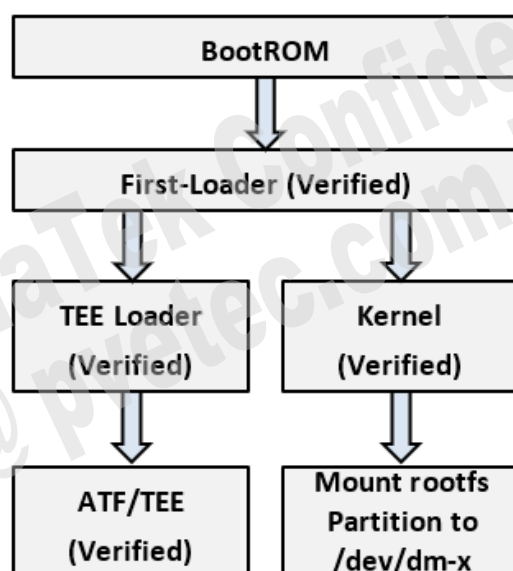


图 1-1. 安全启动检查流程

1.3 下载代理认证 (DAA)

当外部存储器（eMMC/NAND/...）中的设备软件为空或损坏时，主机 PC 工具可以与 SoC 中的 BROM 通信，将一个 DA 软件加载到 SoC 的 SRAM 中并执行 DA 来进行镜像下载过程。当 Enable_DAA 被烧录时，DA 将由 BROM 进行认证，下面的流程被称为下载代理认证（DAA）。

1. 主机 PC 工具通过 USB 或 UART 将认证文件（AuthFile）发送给 BROM。
2. BROM 对 AuthFile 进行认证。

- (1) BROM 从 AuthFile 中读取安全启动检查 (SBC) 公钥 (SBC_PUBK)，并使用公钥哈希 (SBC_PUBK_HASH~SBC_PUBK_HASH1) 及相应的禁用位 (SBC_PUBK_HASH_DIS~SBC_PUBK_HASH1_DIS) 来认证 SBC 密钥的哈希。
- (2) BROM 使用 SBC_PUBK 来验证 AuthFile。
- (3) BROM 从 AuthFile 中获取 DAA 密钥。
3. 主机 PC 工具发送 DA。
4. BROM 使用 DAA 密钥通过 SHA256/RSA-2048 验证 DA。
5. BROM 跳转到 DA 来执行固件下载过程。

注意:

- SBC_PUBK 和 DAA 密钥可以不同，以提高安全级别。

1.4 安全特性配置

1.4.1 生成密钥对

生成两对密钥 (包括私钥和公钥)，SBC_KEY 和 VERIFIED_KEY，并放到 meta/meta-mediatek/conf/machine/keys 目录下。

1. 生成 Yocto 私钥的命令:

```
openssl genrsa -F4 -out sbc_key.pem 2048
openssl genrsa -F4 -out verified_key.pem 2048
openssl req -batch -new -x509 -key verified_key.pem -out verified_key.crt
```

2. 生成公钥的命令:

```
openssl rsa -in sbc_key.pem -pubout > sbc_pubk.pem
openssl rsa -in verified_key.pem -pubout > verified_pubk.pem
```

使用相同的方法生成用于签名和验证 DA 的 DA 密钥对 (da_prvk.pem/da_pubk.pem)，DA 可以和 SBC key 一样，也可以不一样。

1.4.2 开启安全启动

1.4.2.1 生成 SCB pubk HASH

1. 工具路径:

meta/meta-mediatek/recipes-bsp/lk/files/pbp

2. 命令:

```
chmod 777 der_extractor
python pbp.py -j sbc_key.pem -func keyhash_pss -o keyhash
```

1.4.2.2 Blowing SBC_PUBK0_HASH_Field

Step 1: 使用 “hexdump -C keyhash”, or “xxd -c 32 keyhash” 来显示 16 进制 keyhash。

注意:

- 您**必须**使用 -c 这个入参才能生成规范的十六进制的 ASCII 显示结果。

```
hexdump -C keyhash
```

```
00000000 16 b1 oe fc 5e 4e 06 76 e9 d9 6e 40 0c 51 ca 36 | .....
00000010 d1 be 93 d2 67 fd 3e af db f6 f7 89 4a 2c 40 18 |.....
```

```
xxd -c 32 keyhash
```

```
00000000 16b1 oefc 5e4e 0676 e9d9 6e40 0c51 ca36 d1be 93d2 67fd 3eaf dbf6 f789 4a2c
4018      .....
```

禁止使用 hexdump keyhash 来显示。

```
hexdump keyhash
```

```
00000000 b116 fc0e 4e5e 7606 d9e9 406e 510c 36ca
00000010 bed1 d293 fd67 ad3e f6db 89f7 2c4a 1840
```

Step 2: 将结果转化为十六进制的字符串。删除字符之间的空格:

```
16b10efc5e4e0676e9d96e400c51ca36d1be93d267fd3eafdbf6f7894a2c4018
```

Step 3: 执行 ewriter 命令:

```
ewriter 1 0 32 16b10efc5e4e0676e9d96e400c51ca36d1be93d267fd3eafdbf6f7894a2c4018
```

“ewriter”工具更详细的用法，可以参考 *MTK_eFuse_Writer_User_Guide*。

注意:

- 开启 IC 的 BROM secure boot 校验功能，除了要写上述的 SBC_PUBK0_HASH 栏位外，还需要写 SBC_EN。efuse 只能写一次，所以在项目开发阶段不建议写 efuse 开启 IC 的 BROM secure boot 校验功能。

1.4.2.3 软件开启 Secure Boot 配置

```
meta/meta-mediatek-mt8xxx/conf/machine/[project].conf
```

```
SECURE_BOOT_ENABLE = "yes"
```

```
ENABLE_ROOTFS_CHECK = "yes"
```

此配置公版默认开启可以在不使用 eFuse 的情况下启用软件层面安全启动和安全下载，以便验证 LK 以上软件的安全启动流程。

1.4.3 编译软件

编译整个项目。

1.4.4 独立远程签名环境

MTK SDK 支持独立的签名环境。在这种模式下，只能将公钥放入代码库中，稍后再进行最终的签名操作。

1.4.4.1 独立签名环境准备

Step 1: 开启远程独立签名的配置，STANDALONE_SIGN_PREPARE 开启后签名的动作会放到独立的签名环境里面，所以开启此宏后，编译出来的 image 默认是没有签名的。

```
SECURE_BOOT_ENABLE = "yes"
```

```
STANDALONE_SIGN_PREPARE = "yes"
```

```
ENABLE_ROOTFS_CHECK = "yes"
```

Step 2: 保留 verified_pubk.pem 文件在 meta/meta-mediatek/conf/machine/keys 目录，其他密钥放到独立的签名环境。

Step 3: 编译镜像

1.4.4.2 签名镜像

独立的签名环境的工具包位于：meta/meta-mediatek-mt86XX/recipes-devtools/standalone_sign_env，客户可以根据自己的需要可以将工具包放到特定的服务器上。（如果想进行远程签名也是支持的，另有远程签名的文档可以参考）

Step 1:

将相关密钥\${SBC_KEY}.pem, \${VERIFIED_KEY}.pem 放到“keys”文件夹下。

Step 2: 执行签名脚本: sh sign_images.sh

如果一切正常，签名的镜像将被放置在“signed”文件夹中。

注意:

- sign_images.sh 是 MTK SDK 的示例文件。强烈建议您使用该文件并根据您的需要进行修改。

1.5 签名 DA

下载代理验证 (DAA) 是 MediaTek 的解决方案，可确保固件下载进度的安全。在开始身份验证之前，请确保 SBC 已启用并且安全启动测试已通过。Yocto 用的 Android 这边的 DA，所以在签名 DA 时会用到 Android 相关的脚本和源码。

1.5.1 生成 dakey.h

将 DA 公钥 (da_pubk.pem) 导出，使用 der_extractor 生成 dakey.h, 该工具位于 Android 源码

vendor/mediatek/proprietary/scripts/sign-image_v2/der_extractor/ 目录下，请将生成的 dakey.h 放置到以下 Yocto 路径：

```
[LK2] $LK2/target/$PROJECT/include/dakey.h
```

命令:

```
chmod 777 der_extractor
python pem_to_der.py da_pubk.pem da_pubk.der
./der_extractor da_pubk.der dakey.h ANDROID_SBC
```

1.5.2 签名 DA

1. 密钥路径设置

将 DA 公钥 (da_prvk.pem) 导出, 放置到 Android 路径

vendor/mediatek/proprietary/scripts/secure_chip_tool/custom_keys 目录下。

2. 将需要签名的 DA 放置在 prebuilt/resignda/ 目录下, 并执行以下命令来签名 DA:

```
cd vendor/mediatek/proprietary/scripts/secure_chip_tool/  
python MTK/resign_da.py prebuilt/resignda/DA_BR.bin MT6897  
settings/Legacy/da/bbchips_pss.ini all out/resignda/DA_BR-resing.bin
```

1.5.3 生成 Authfile

DA 的公钥包含在 authfile 中, BROM 使用它来验证 DA。因此, 如果启用了 DAA, 在使用 flashtool 下载镜像时需要 authfile。所有位于以下位置 (android branch) 的 .ini 文件, 已经为本项目配置好了。

./vendor/mediatek/proprietary/scripts/secure_chip_tools/settings/Legacy/authfile/

您只需要替换位于以下位置的 DA、和根私钥 (将 sbc_key.pem 重命名为 root_prvk.pem) 的 .pem 文件。

./vendor/mediatek/proprietary/scripts/secure_chip_tools/custom_keys/

Authfile 生成命令:

```
python MTK/toolauth.py -i settings/Legacy/authfile/toolauth_key.ini -g  
settings/Legacy/authfile/toolauth_gfh_config_pss.ini out/toolauth/auth_sv5.auth
```

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。